

untangle::executor

Generated by Doxygen 1.18.0

1	executor	1
2	Namespace Index	3
2.1	Namespace List	3
3	Class Index	5
3.1	Class List	5
4	File Index	7
4.1	File List	7
5	Namespace Documentation	9
5.1	untangle Namespace Reference	9
5.1.1	Detailed Description	9
6	Class Documentation	11
6.1	untangle::executor Class Reference	11
6.1.1	Detailed Description	11
6.1.2	Member Typedef Documentation	12
6.1.2.1	task_t	12
6.1.3	Constructor & Destructor Documentation	12
6.1.3.1	executor()	12
6.1.3.2	~executor()	12
6.1.4	Member Function Documentation	13
6.1.4.1	submit()	13
7	File Documentation	15
7.1	executor.hpp	15
	Index	19

Chapter 1

executor

c++ thread pool executor based on async execution

The interface is one header that exposes a pool of workers, each one an **async** execution in continuous mode, fed by a shared queue of tasks.

This is the prototype, imported. It was written in `async/prototypes/` to answer whether execution is a usable building block for a pool — it is — and it is being hardened here. The behaviour below is what the tests state today, not a settled interface.

requirements

C++23, and the `async` submodule:

```
git submodule update --init --recursive
```

`async` carries its own actuator submodule, which is why the clone has to be recursive. Both are header-only; nothing is built or installed.

The headers report warnings with `std::println`, so they need a standard library that provides `<print>` — GCC 14 or newer.

example

```
#include <executor.hpp>

int main() {
    untangle::executor pool(4);

    for (int i = 0; i < 100; ++i) {
        pool.submit([i] { work(i); });
    }

    return 0; // ~executor drains what was submitted, then stops the workers
}
```

how work is placed

N executions run in continuous mode behind a shared `std::deque` of tasks.

Nothing overtakes. A task goes straight to a free worker only when the shared queue is empty; otherwise it joins the back of the queue. Finding a free worker does not let a task jump a queue that already has work in it.

Workers pull rather than being pushed to. When one reports its queue drained — `on_finished`, raised by execution the moment its list empties — it takes the front of the shared queue. That is what spreads the work, with no scheduling logic of its own: 400 tasks over 4 workers come out 100/100/100/100.

A worker is asked, not tracked. `execution::is_busy()` is what `submit()` reads to find a free worker and what the destructor reads to decide the pool is idle. A tally kept here would be the pool's belief about its workers; this is the workers' own answer, and it cannot drift.

lifetime and failure

The destructor drains. It refuses further work, waits for the queue to empty and for every worker to go idle, and only then stops them. A task submitted before the pool goes out of scope has run by the time it does.

`submit()` says whether the task was taken. It returns false once the pool is shutting down, which a task submitting more work from inside the pool can see.

A task that throws does not take the worker with it. `execution` catches it, reports it on `stderr`, and the worker carries on with the next task. Nothing reaches the submitter — a task that has something to report carries its own channel, because a continuous worker does not collect results.

building the tests

```
cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build
```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access.

sanitizers

`EXECUTOR_SANITIZE` builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DEXECUTOR_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are `address`, `thread`, `undefined`, or empty. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. `address` and `thread` instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

the reference

`tools/make_doc.sh` builds `doc/refman.pdf` from the header and this README. It needs doxygen, tectonic and python3, and it fails on any doxygen warning that is not an `obsolete-tag` notice, so an undocumented member or a broken `\ref` does not reach the reference.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	A thread pool executor built out of <code>untangle::async::execution</code> objects	9
--------------------------	---	-------------------

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::executor	Runs tasks on a fixed pool of <code>untangle::async::execution</code> workers	11
------------------------------------	---	--------------------

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

executor.hpp	15
--	----

Chapter 5

Namespace Documentation

5.1 untangle Namespace Reference

A thread pool executor built out of `untangle::async::execution` objects.

Classes

- class `executor`
Runs tasks on a fixed pool of `untangle::async::execution` workers.

5.1.1 Detailed Description

A thread pool executor built out of `untangle::async::execution` objects.

Imported from the `async` repo's `prototypes/executor.hpp`, unchanged apart from its namespace and this comment. The hardening the prototype's README asks for has not been done yet.

The pool holds `N` executions in continuous mode and a shared queue of tasks. A task is handed straight to a free worker only when the shared queue is empty; otherwise it goes to the back of the queue, and a worker takes the front of the queue as it frees up. Nothing overtakes: finding a free worker does not let a task jump a queue that already has work in it.

Chapter 6

Class Documentation

6.1 untangle::executor Class Reference

Runs tasks on a fixed pool of untangle::async::execution workers.

```
#include <executor.hpp>
```

Public Types

- using `task_t` = `std::function<void(void)>`

Public Member Functions

- `executor` (`std::size_t worker_count`)
Starts `worker_count` executions in continuous mode.
- `~executor` ()
Drains what has been submitted, then stops every worker.
- `bool submit` (`task_t task`)
Queues a task, or hands it to a worker that has nothing to do.
- `std::size_t pending` () `const`
How many tasks are waiting for a worker.

6.1.1 Detailed Description

Runs tasks on a fixed pool of untangle::async::execution workers.

Remarks

Tasks are `std::function<void(void)>`. An execution's action type is fixed by its template argument, so one pool runs one signature; a task that needs to return something carries its own channel, because a continuous worker does not collect results.

6.1.2 Member Typedef Documentation

6.1.2.1 task_t

using `untangle::executor::task_t` = `std::function<void(void)>`

The work a pool runs. One pool runs one signature, because an execution's action type is fixed by its template argument.

6.1.3 Constructor & Destructor Documentation

6.1.3.1 executor()

```
untangle::executor::executor (
    std::size_t worker_count) [inline], [explicit]
```

Starts `worker_count` executions in continuous mode.

Parameters

<code>worker_count</code>	- How many workers to run. Must not be zero.
---------------------------	--

Exceptions

<code>std::invalid_argument</code>	- <code>worker_count</code> is zero. Such a pool would accept work and then have nobody to run it: <code>submit()</code> would queue the task and answer true, and the destructor would wait for a queue only a worker can empty. It is refused before any worker is started, so a pool that cannot run anything never exists rather than existing and hanging.
------------------------------------	---

Remarks

`std::thread::hardware_concurrency()` returns 0 when it cannot tell how many cores there are, so a caller passing it straight through is the way this is most likely to be reached. Choosing a default in that case is the caller's to make, not the pool's.

6.1.3.2 ~executor()

```
untangle::executor::~executor () [inline]
```

Drains what has been submitted, then stops every worker.

Remarks

Every worker has left its thread before the first execution is destroyed. That is what the poll is for: `~execution()` waits only for its own worker, so destroying the executions one at a time would free the first while another's thread is still reading it through `nothing_running()`. The wait is on this pool's own poll, so a second pool running alongside this one is not something this destructor waits for.

6.1.4 Member Function Documentation

6.1.4.1 submit()

```
bool untangle::executor::submit (  
    task\_t task) [inline]
```

Queues a task, or hands it to a worker that has nothing to do.

Returns

true - the task was accepted. false - the pool is shutting down and refused it.

The documentation for this class was generated from the following file:

- executor.hpp

Chapter 7

File Documentation

7.1 executor.hpp

```
00001 // Copyright (c) 2026 Nicolae Popescu. MIT License.
00002
00014 #pragma once
00015
00016 #include <async.hpp>
00017 #include <chrono>
00018 #include <condition_variable>
00019 #include <cstdint>
00020 #include <deque>
00021 #include <functional>
00022 #include <memory>
00023 #include <mutex>
00024 #include <stdexcept>
00025 #include <string>
00026 #include <thread>
00027 #include <utility>
00028 #include <vector>
00029
00030 namespace untangle {
00031
00039 class executor {
00040 public:
00043     using task_t = std::function<void(void)>;
00044
00059     explicit executor(std::size_t worker_count) {
00060         if (worker_count == 0) {
00061             throw std::invalid_argument("executor: worker_count must not be zero");
00062         }
00063
00064         workers_.reserve(worker_count);
00065
00066         for (std::size_t index = 0; index < worker_count; ++index) {
00067             auto next = worker_execution::create_instance("pool_worker_" + std::to_string(index));
00068
00069             // How a worker learns there is more waiting for it. is_busy() answers whether a worker has
00070             // room; this is what tells the pool the moment one frees up, without anybody polling.
00071             next->on_finished = [this, index] { take_next_task(index); };
00072
00073             // A worker runs in a detached thread and cannot be joined, so asking whether it is still
00074             // running is the only way to wait for one, and the poll answers that for any number of them
00075             // at once. This pool's own poll rather than the process-wide one: what the destructor asks
00076             // is about these workers, and execution_poll::get() would answer it for every execution in
00077             // the process. Nothing here has to unregister - the record runs both ways, so whichever of
00078             // the two dies first takes itself out of the other.
00079             poll_.add(*next);
00080
00081             workers_.push_back(std::move(next));
00082             workers_.back()->start();
00083         }
00084     }
00085
00095     ~executor() {
00096     {
00097         std::unique_lock<std::mutex> lock(mutex_);
00098         accepting_ = false;
00099         drained_cv_.wait(lock, [this] { return pending_.empty() && nothing_running(); });
00100     }
00101 }
```

```

00101
00102 // Outside the lock: stop() wakes each worker, and a worker waking up runs on_finished, which
00103 // wants this mutex.
00104 for (auto& one : workers_) {
00105     one->stop();
00106 }
00107
00108 // The poll stands in for a join. Until it reports idle, a worker can still be inside
00109 // take_next_task(), reading every execution through nothing_running() and is_busy() - which is
00110 // what makes destroying them here a use-after-free rather than a teardown.
00111 while (poll_.is_running()) { // time of check
00112     std::this_thread::sleep_for(std::chrono::milliseconds(poll_interval_ms));
00113 }
00114
00115 // time of use: no worker thread is left to reach workers_, so clearing it races nothing.
00116 workers_.clear();
00117 }
00118
00124 bool submit(task_t task) {
00125     std::lock_guard<std::mutex> lock(mutex_);
00126
00127     if (!accepting_) {
00128         return false;
00129     }
00130
00131     // The queue comes first whenever it has anything in it: a task that arrives while others are
00132     // waiting does not get to overtake them just because a worker happens to be free.
00133     if (pending_.empty()) {
00134         for (std::size_t index = 0; index < workers_.size(); ++index) {
00135             if (!workers_[index]->is_busy()) {
00136                 give_to_worker(index, std::move(task));
00137                 return true;
00138             }
00139         }
00140     }
00141
00142     pending_.push_back(std::move(task));
00143     return true;
00144 }
00145
00149 std::size_t pending() const {
00150     std::lock_guard<std::mutex> lock(mutex_);
00151     return pending_.size();
00152 }
00153
00154 private:
00155     using worker_execution = untangle::async::execution<std::function<void(void)>>;
00156
00163 void give_to_worker(std::size_t index, task_t task) {
00164     // add_action() takes the execution's own action_mutex while this holds mutex_. That is only
00165     // safe in one direction, and it holds: a worker calls on_finished with action_mutex released,
00166     // so it never takes mutex_ while holding action_mutex, and the two never form a cycle.
00167     workers_[index]->add_action(std::move(task));
00168 }
00169
00173 void take_next_task(std::size_t index) {
00174     std::lock_guard<std::mutex> lock(mutex_);
00175
00176     if (!pending_.empty()) {
00177         task_t next = std::move(pending_.front());
00178         pending_.pop_front();
00179         give_to_worker(index, std::move(next));
00180         return;
00181     }
00182
00183     if (nothing_running()) {
00184         drained_cv_.notify_all();
00185     }
00186 }
00187
00194 bool nothing_running() const {
00195     for (const auto& one : workers_) {
00196         if (one->is_busy()) {
00197             return false;
00198         }
00199     }
00200
00201     return true;
00202 }
00203
00206 static constexpr int poll_interval_ms = 50;
00207
00208 mutable std::mutex mutex_;
00209 std::condition_variable drained_cv_;
00210
00222 async::execution_poll poll_;
00223

```

```
00224  std::deque<task_t> pending_;
00225  std::vector<std::shared_ptr<worker_execution> > workers_;
00226  bool accepting_ = true;
00227 };
00228
00229 } // namespace untangle
```


Index

- ~executor
 - untangle::executor, [12](#)
- executor, [1](#)
 - untangle::executor, [12](#)
- submit
 - untangle::executor, [13](#)
- task_t
 - untangle::executor, [12](#)
- untangle, [9](#)
- untangle::executor, [11](#)
 - ~executor, [12](#)
 - executor, [12](#)
 - submit, [13](#)
 - task_t, [12](#)